GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Sesión 06 — Patrones de Diseño

2º CFGS DAW/DAM · Proyecto Intermodular II

Autores: Sergi García, Guillermo Garrido y Alfredo Oltra

🗺️ La sesión en un mapa

De crear objetos a organizar sistemas: **soluciones probadas y reutilizables** a los problemas comunes del diseño de software.

⚙️

Creacionales

Singleton · Factory · Builder

🧩

Estructurales

Adapter · Facade · Proxy

📡

Comportamiento

Observer · Strategy · Iterator

🏛️

MVC · MVVM

Organizar la app

♻️

Arquitectura Limpia

Hexagonal

🚀

Microservicios

BFF · API · Eventos

ℹ️ Todo se aplicará al proyecto **TaskFlow**: cada patrón resuelve un problema concreto que vais a tener este curso.

🧠 ¿Qué es un patrón de diseño?

No es código para copiar y pegar (ni una función tipo `sort()`): es una **solución probada y reutilizable** a un problema común.

🍳 Algoritmo = receta

Pasos exactos y secuenciales: «1. Cortar cebolla · 2. Sofreir · 3. Añadir tomate».
Si te saltas un paso, **fallas**.

📐 Patrón = plano

Te muestra el resultado y la disposición de las piezas.
Los materiales y el orden **los decides tú** en cada proyecto.

ℹ️ Un patrón es una **plantilla conceptual**: una idea estructurada, no una pieza de código.

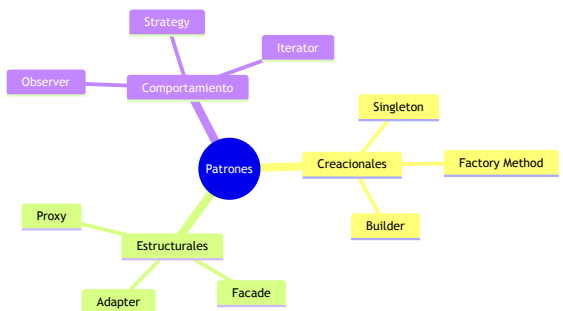
💪 ¿Por qué usar patrones?

Beneficio	Sin patrones	Con patrones
Lenguaje común	«Necesito una clase con una única instancia que centralice la conexión a la BD...»	«Usa un **Singleton** para la conexión a la BD»
Soluciones comprobadas	Reinventar la rueda (y sus errores)	Décadas de uso en la industria, diseño sólido (SOLID)
Mantenibilidad	Código acoplado: cambia una pieza, se rompe otra	Desacopla el sistema: escala y se modifica sin romperse

⚠️ Una mala arquitectura genera **deuda técnica** que colapsa el proyecto cuando crece: los patrones son una inversión en el futuro de vuestro código.

🗂️ Las tres familias

Familia	Propósito	Analogía clave
Creacionales	Cómo se **crean** los objetos, sin usar `new` directamente	🏭 La fábrica: delegar la producción
Estructurales	Cómo se **ensamblan** en estructuras mayores	🔌 El adaptador: conectar lo incompatible
Comportamiento	Cómo **colaboran** y se reparten responsabilidades	📻 La radio: emitir sin saber quién escucha



⚙️ Creacionales (I): Singleton

Problema: ciertas cosas solo pueden existir **una vez** (conexión a la BD, configuración, logger). Si cada módulo crea la suya... caos.

> ***Analogía*** 🏀***:*** *en un videojuego de baloncesto solo hay* ***una pelota****. El patrón garantiza esa única instancia y un punto de acceso global a ella.*

```
public class BasedeDatos {
  private static BasedeDatos instancia;   // la única

  private BasedeDatos() { }               // constructor privado: nadie hace new()

  public static BasedeDatos getInstancia() {
    if (instancia == null) instancia = new BasedeDatos();
    return instancia;
  }
}
```

ℹ️ Uso razonable: configuración, caché, conexión central. **Abuso**: convierte al Singleton en un contenedor global de todo («God Object») difícil de testear.

⚙️ Creacionales (II): Factory y Builder

🏭 Factory Method

Delegar *qué objeto* crear a una fábrica, según el contexto.

Ejemplo: la fábrica decide si crea `Barco` o `Camion` según la ruta; el conductor no sabe detalles de fabricación.

🍔 Builder

Construir objetos complejos **paso a paso** con opciones.

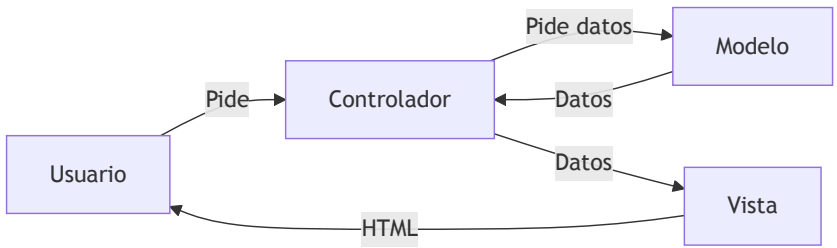
Ejemplo: pedir una hamburguesa a medida: sin cebolla, doble queso, pan brioche. Cada opción, un paso del builder.

ℹ️ Ambos combaten lo mismo: el `new` por todas partes («hard-coding») que obliga a reescribir el código cuando cambia una clase.

🏛️ MVC: el clásico

Separar la interfaz de la lógica. Estándar en backend (Laravel, Spring, Django).

> ***Analogía 🍽️ El restaurante:*** *la* ***Vista*** *es el cliente que mira la carta; el* ***Controlador*** *es el camarero que toma nota y coordina (no cocina); el* ***Modelo*** *es la cocina: ingredientes, datos y reglas.*



1. Router: recibe la URL y elige el controlador.
2. Controlador: pide datos al modelo.
3. Vista (plantilla Blade/Twig): recibe los datos y genera el HTML.

🏛️ MVC en Laravel (DAW)

```php
<?php
namespace App\Http\Controllers;
use App\Models\User;            // el MODELO

class UserController extends Controller {
    // El CONTROLADOR recibe la petición
    public function show($id) {
        $user = User::find($id);            // habla con el modelo
        return view('user.profile',        // carga la VISTA
            ['user' => $user]);
    }
}
```

```blade
←!— resources/views/user/profile.blade.php (la VISTA) —→
<h1>Perfil de {{ $user→name }}</h1>
<p>Email: {{ $user→email }}</p>
```

ℹ️ El controlador **no** devuelve HTML: devuelve una vista que se renderiza. Cada cosa en su sitio.

🏛️ MVVM: el moderno

Model–View–ViewModel, hecho para pantallas reactivas: móvil (Jetpack), React, Vue.

> ***Analogía 🪞 El espejo:*** *el ViewModel «sonríe» y la vista se actualiza al instante. No hay que decirle «píntame la sonrisa»: la* ***reactividad*** *lo hace sola (data binding).*




```
class ContadorViewModel : ViewModel() {
    val numero = MutableLiveData(0)
    fun incrementar() { numero.value = (numero.value ?: 0) + 1 }
}
// La Vista se SUSCRIBE: cada cambio de 'numero' repinta la UI
viewModel.numero.observe(this) { texto.text = "Valor: $it" }
```

ℹ️ La vista **no conoce al modelo**: solo al ViewModel. En React/Vue, el `state` del componente cumple ese mismo papel (`UI = f(State)`).

🛡️ Estructurales: Adapter y Facade

Problema: clases incompatibles, o añadir funcionalidad sin tocar el código fuente.

🔌 Adapter

El adaptador de viaje: envuelve tu enchufe para que encaje en la pared. Ni la pared ni el portátil cambian, pero **ahora cooperan**.

Caso real: una API antigua que devuelve **XML** funcionando en tu app moderna que espera **JSON**.

🏨 Facade

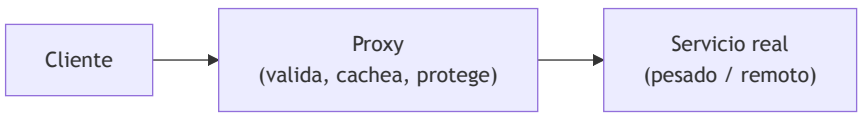
La recepción del hotel: una interfaz simple para manejar sistemas complejos internos (limpieza, cocina, reservas) que **no ves**.

Caso real: `CamaraFacil().tomarFoto()` que por dentro gestiona permisos, sensores y buffers.

🛡️ Estructural: Proxy

Un **intermediario que controla el acceso** al objeto real, añadiendo lógica sin tocarlo.

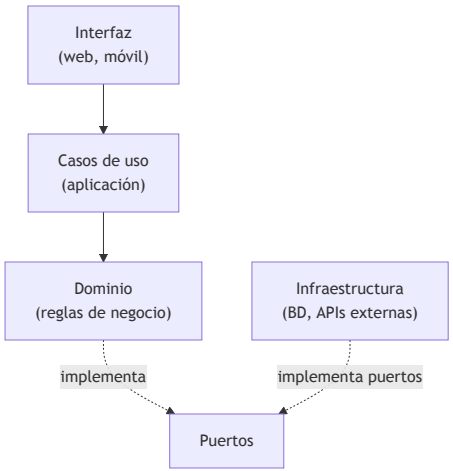
> ***Analogía 💳 La tarjeta de crédito:*** *representa el dinero real (que está en el banco) y añade* ***control, validación y seguridad*** *antes de completar la transacción.*



1. El cliente llama al proxy igual que llamaría al objeto real (misma interfaz).
2. El proxy decide: valida permisos, devuelve caché o reenvía al real.
3. El objeto real queda protegido: nadie lo usa sin pasar por el proxy.

♻️ Arquitectura Limpia (Hexagonal)

El dominio en el centro: la lógica de negocio no depende de la BD, del framework ni del UI.



Ventaja	Consecuencia
El núcleo es **testeable** sin BD ni HTTP	Tests rápidos y estables
Cambiar de BD o de UI solo toca la capa exterior	Menos riesgo al evolucionar
Los «puertos» son los contratos	Adapters intercambiables

⚠️ La idea clave: **las dependencias apuntan hacia el dominio**. Si tu clase de negocio importa una librería de base de datos, la arquitectura está invertida.

🚀 Microservicios y BFF

De un solo bloque a **servicios pequeños e independientes**, cada uno con su BD.

🧱 Monolito

Todo en una app y una BD.
+ Simple de desarrollar y desplegar
– Escalar = escalar todo; acoplamiento

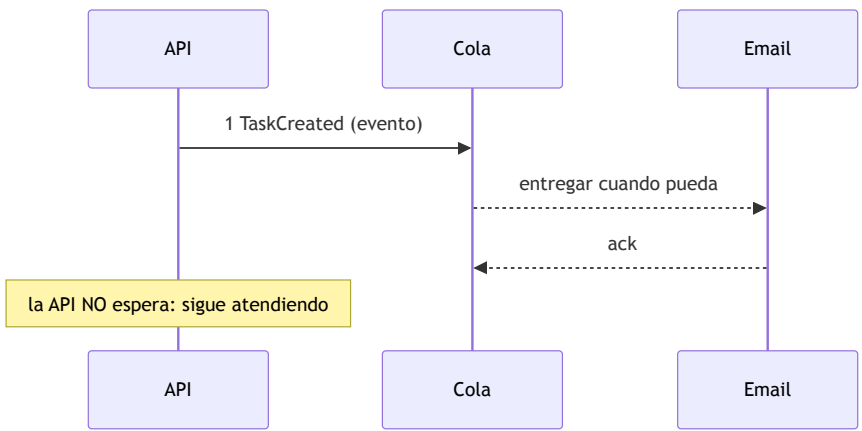
📱 BFF *(Backend For Frontend)*

Una capa intermedia por cliente: el BFF de móvil devuelve datos ya recortados para la pantalla del móvil; el BFF de web, para la web.
+ Cada cliente recibe lo que necesita
– Más piezas que mantener

ℹ️ En TaskFlow: una API principal (DAW) y, si el móvil (DAM) necesita datos muy distintos, un BFF móvil delante. No empieces por microservicios: **gana primero un monolito modular**.

📡 ¿Cómo hablan los servicios?

	Síncrono ☎️ (llamada)	Asíncrono ✉️ (email)
Modelo	«Te pregunto y **me quedo esperando**»	«Te dejo el mensaje en el buzón, lo lees cuando puedas»
Tecnologías	REST, GraphQL, gRPC	Colas de mensajes: RabbitMQ, Kafka
Riesgo / baza	Si el receptor está caído, **tú también fallas**	Si el receptor está caído, el mensaje **espera en la cola**: resiliencia



ℹ️ Comunicación por **eventos** («TaskCreated») desacopla productores y consumidores: el emisor no sabe quién escucha, como una **radio** 📻. El correo de bienvenida de TaskFlow debe ir asíncrono.

📦 Patrones de acompañamiento: Sidecar y Ambassador

Cuando el patrón vive **fuera del proceso**, a su lado (*sidecar*) o delante (*ambassador*). Muy usados en contenedores y Kubernetes.

🛵 Sidecar

Un proceso **a tu lado**, mismo ciclo de vida: ej. un contenedor de logs al lado de la app en el mismo Pod.

Gana: la app no sabría exportar logs; el sidecar los colecta.

🚀 Ambassador

Un **delegado en la red**: la app llama a «localhost», el ambassador hace retry, TLS o descubrimiento de servicios.

Gana: la lógica de red vive fuera del código de la app.

ℹ️ Ambos aplican la misma idea hexagonal: **separar la lógica de negocio de la infraestructura** sin tocar el código de la aplicación.

🛡️ Resiliencia y seguridad (patrones de la gran ciudad)

Patrón	Qué hace	Analogía ciudad
Circuit Breaker	Si el servicio falla mucho, «abre el circuito» y no lo llama un tiempo	🔌 Un fusible: corta la corriente
Retry + Backoff	Reintentar con esperas crecientes: 1s, 2s, 4s…	🚦 Volver a intentarlo cuando la cola baja
Rate Limiting	Limitar cuántas peticiones acepta un servicio	🎫 Entradas agotadas: no entra más gente
API Gateway	Un único punto de entrada: autentica, limita y enruta	🛂 El control del aeropuerto

⚠️ En TaskFlow: el **API Gateway** delante de los servicios y un **circuit breaker** llamando al servicio de email externo: si SendGrid cae, TaskFlow no debe caer con él.

🧩 Aplicación al proyecto: TaskFlow con patrones

Necesidad del proyecto	Patrón que la resuelve
Una única conexión a la BD y configuración global	**Singleton**
Crear notificaciones push/email según el canal	**Factory Method**
Adaptar la API antigua XML al cliente nuevo JSON	**Adapter**
Simplificar el subsistema de BD en un método `guardarTarea()`	**Facade**
Caché y control de acceso delante de la API	**Proxy**
Notificar a los clientes cuando cambia una tarea	**Observer / eventos** (asíncrono)

ℹ️ No os vale «conozco 30 patrones»: en la memoria, **justificad 3-4 patrones concretos** que hayáis aplicado, con el problema y la solución.

✅ Resumen de la sesión

- Un patrón es una **solución probada y reutilizable**: un plano, no un código para copiar.
- **Creacionales** (Singleton, Factory, Builder) controlan los `new`.
- **Estructurales** (Adapter, Facade, Proxy) conectan y protegen piezas.
- **MVC/MVVM y Arquitectura Hexagonal** organizan la app: cada cosa en su sitio, dependencias hacia el dominio.
- **Microservicios, eventos y patrones de resiliencia** preparan el sistema para crecer sin caerse.
- En la memoria del proyecto: **3-4 patrones aplicados, con problema y justificación**.

ℹ️ Todos los patrones comparten una obsesión: **reducir el acoplamiento** — que un cambio en una pieza no rompa las demás.

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //